Davante

Práctica 1

Entornos de Desarrollo
1º Desarrollo de Aplicaciones Multiplataforma

Profesorado: Lorenzo Toro

Curso 2026/2027

Trabajo presentado por:
DANIEL PERERA MARTÍN

Índice

Práctica 1: Pseudocódigo y paradigmas3

1. El caso3
2. Actividades7

Actividad 1: La calculadora de medias que no se rompe7

Actividad 2: Un algoritmo con bucle: el contador de tareas9

Actividad 3: Los paradigmas de programación11

3. Entrega14

Práctica 1: Pseudocódigo y paradigmas

1. El caso

La clase va a estrenar la **app de gestión del aula**, un pequeño programa para que el delegado lleve la cuenta de las tareas del grupo y las notas de los retos.

Antes de escribir ni una línea de código hay que **pensar el programa**: qué datos entran, qué pasos se siguen y qué sale por pantalla.

Se debería pensar en los siguientes datos entrantes. Las variables enteras:

- tarea -> para contabilizar la cantidad de tareas que dispone el grupo. Se presupone que tarea que está apuntada es que no está realizada.
- nota -> con ella añadiríamos la calificación que se espera de cada ejercicio o reto que se haya realizado en clase. Gracias a esta variable debemos pensar en que el alumnado seguramente querrá la media de las calificaciones que va haciendo para saber si debe o no esforzarse más en el estudio. Para ello se necesitarán las variables necesarias para calcular la media: cantidadNotas y sumaNotas (que guardará la suma de la variable nota para posteriormente usarla en el algoritmo necesario para resolver el problema)
- opcion -> la forma de pensar en una app simple es hacer un menú simple que pueda interactuar el usuario. Por ello, he pensado en mostrar un texto que disponga las diferentes opciones que puede elegir.

Acto seguido le daremos (como suelo entenderlo) un valor de “pizarra limpia”, es decir, le otorgaremos a cada variable el valor 0 para que posteriormente vaya registrando el ir y venir de la contabilidad de las variables ya mencionadas.

Como indiqué anteriormente se mostraría un “menú” con las diferentes opciones. Dado mi escueto conocimiento de programación y con los contenidos dados actualmente creo poder vislumbrar la posibilidad de realizarlo de la siguiente manera.

Con un bucle MIENTRAS podemos crear un coto donde el usuario hasta que pulse la opción 6 navegará por las opciones secuenciadas en condicionales SI y SI NO SI, tales como:

- 1. Añadir tarea -> permitirá sumar 1 tarea pendiente al conteo.
- 2. Quitar tarea -> permitirá restar 1 tarea, dando por hecho que se ha logrado.
- 3. Ver tareas actuales -> permitirá que el usuario vea la cantidad de tareas pendientes. Si no hay ninguna porque el resultado sea 0 se mostrará la imposibilidad de quitar tarea.
- 4. Añadir nota de reto -> se podrá añadir una nota dentro del rango clásico de 0 y 10. Si no se cumple con el rango se notificará al usuario que es inválido y que inserte una nota dentro del ámbito acordado. Si añade una nota dentro del rango se acumulará su calificación en la variable sumaNotas y se añadirá un +1 a la variable cantidadNotas que almacenará la cantidad de notas a disposición para hacer la siguiente acción.
- 5. Media de notas -> aquí usaremos las variables ya mencionadas de cantidadNotas y sumaNotas para realizar la operación en una nueva variable (llamada media) que calcule la media aritmética. A su vez se contemplará la posibilidad de que el usuario podría pulsar esta opción sin tener calificación alguna. Esto creará un mensaje advirtiendo que no hay calificaciones para calcular.
- 6. Salir -> la opción que nos hace salir del bucle y finalizar así el programa. Con ella mostraremos un mensaje de despedida a modo de señal de finalización.
- Por último y no menos importante, hay que discernir la posibilidad de que el usuario introduzca un número que no esté en dentro del menú. Para ello el final del último condicional SI NO, solventaría esta posibilidad mostrando un mensaje de opción inválida y retornando al inicio del ciclo.

El ejemplo del código sería el siguiente:



INICIO

DEFINIR tareas, sumaNotas, cantidadNotas, nota, opcion = entero

tareas = 0

sumaNotas = 0

cantidadNotas = 0

nota = 0

opcion = 0

MIENTRAS opcion NO 6

MOSTRAR Menu:

1. Añadir tarea
2. Quitar tarea
3. Ver Tareas actuales
4. Añadir nota de reto
5. Media de notas
6. Salir

LEER opción

SI opcion = 1

tareas = tareas + 1

SI NO SI opcion = 2

SI tareas > 0

tareas = tareas - 1

SI NO

MOSTRAR "No se puede quitar. No hay ninguna tarea."

SI NO SI opcion = 3

MOSTRAR "Tareas actuales: " + tareas

SI NO SI opcion = 4

LEER nota

SI nota >= 0 Y nota <= 10

sumaNotas = nota + sumaNotas



```
                    cantidadNotas = cantidadNotas + 1
            SI NO
                    MOSTRAR     “Nota inválida. Por favor inserte una nota entre 0 – 10.”
    SI NO SI        opcion = 5
            SI      cantidadNotas <= 0
                    MOSTRAR     “No hay notas”
            SI NO
                    media = sumaNotas / cantidadNotas
                    MOSTRAR     “La media actual es: “ + media
    SI NO SI        opcion = 6
            MOSTRAR     “Saliendo del programa. ¡Hasta luego!”
    SI NO
            MOSTRAR     “Esta opción es inválida”
    FIN SI
FIN MIENTRAS
```

FIN

2. Actividades

Actividad 1: La calculadora de medias que no se rompe

El programa de la app debe calcular **la media de las notas** de un reto. Tu compañero/a escribe este pseudocódigo, pero tiene un fallo: si alguien no tiene notas, la suma es 0 y al dividir entre 0 el programa **se rompe** (o muestra un resultado absurdo):

```
INICIO
    LEER numeroNotas
    suma = 0
    contador = 0

    SI numeroNotas == 0 ENTONCES // quizás se debería cancelar la posibilidad negativa con <=
        MOSTRAR "No existen calificaciones que calcular"
    SI NO
        MIENTRAS contador < numeroNotas HACER
            LEER nota
            suma = suma + nota
            contador = contador + 1
        FIN MIENTRAS
        media = suma / numeroNotas
        ESCRIBIR media
    FIN SI
FIN
```

Se pide:

- Reescribe el pseudocódigo **usando una condicional** para que, si el número de notas es 0 (el divisor), **muestre un mensaje de error al usuario** en lugar de hacer la división.
- Comprueba tu pseudocódigo con la **prueba de escritorio** de los cuatro casos de la tabla. La prueba de escritorio consiste en **ejecutar el algoritmo a mano**, como si tú fueras el ordenador: recorre las instrucciones en orden, anota el valor de las variables y lo que se mostraría en pantalla, tal y como hemos hecho en clase. Es la forma de comprobar si un algoritmo funciona antes de programarlo.

Caso de prueba	¿Qué debe mostrar el programa?	Tu prueba: ¿qué mostró?
3 notas (7, 8, 9)	8	8
2 notas (1, 4)	2,5 (si son Reales)	2,5 (Si son reales)
1 nota (4)	4	4
0 notas	"No existen calificaciones que calcular"	"No existen calificaciones que calcular"

Actividad 2: Un algoritmo con bucle: el contador de tareas

Otra parte de la app **cuenta las tareas del grupo**. El delegado introduce por teclado el estado de cada tarea: **T** si está terminada o **P** si está pendiente. Al final hay que decir cuántas hay de cada tipo.

Escribe el **pseudocódigo completo** de este algoritmo. Debe:

- Leer los estados de las tareas **hasta que** se escriba TERMINAR (bucle).
- Contar cuántas son **T** y cuántas son **P**.
- Al terminar, mostrar el **total de tareas**, las terminadas y las pendientes.

```
INICIO
    DEFINIR     T, P, totalTareas, contadorT, contadorP = entero
    DEFINIR     texto = cadena
    T = 0
    P = 0
    contadorT = 0
    contadorP = 0

    MIENTRAS    texto NO ES “TERMINAR” HACER
            MOSTRAR     “Añade cuántas tareas tienes terminadas: “
            LEER   T
            contadorT = T + contadorT
            MOSTRAR     “Añade cuántas tareas tienes pendientes: “
            LEER   P
            contadorP = P + contadorP
            MOSTRAR     “Tienes “ + contadorT + “ tareas terminadas. ¡Enhorabuena!”
```



```
            MOSTRAR    "Tienes " + contadorP + " tareas pendientes. ¡Ánimo!"
            MOSTRAR    "¿Terminamos? (TERMINAR)"
            LEER   texto
        FIN MIENTRAS
        totalTareas = contadorT + contadorP
        MOSTRAR    "Total de tareas: " , totalTareas,
        MOSTRAR    "Terminadas: " , contadorT
        MOSTRAR    "Pendientes: " , contadotP
    FIN
```

Actividad 3: Los paradigmas de programación

En el tema hemos visto que los **dos grandes enfoques** de los paradigmas son el **imperativo** (decir cómo se hace, paso a paso) y el **declarativo** (decir qué quieres conseguir). Pero no son los únicos: existen **más paradigmas**, y muchos lenguajes actuales son **multiparadigma** porque combinan varios.

Debes investigar **2 de los siguientes 6 paradigmas**: orientado a objetos, funcional, estructurado, lógico, orientado a eventos y concurrente.

Para **cada paradigma elegido**, busca en internet y anota:

- Una **definición con tus palabras** (máximo 4 líneas): qué idea de programación defiende.
- Un **ejemplo de lenguaje** que lo utilice e indica si además lo combina con otros.
- Un **pequeño trozo de código** en ese lenguaje (5-10 líneas) que muestre su estilo.
- Las **direcciones web** donde has encontrado la información: todo lo que afirmes debe poder comprobarse.

Para terminar, responde: ¿el pseudocódigo de las actividades 1 y 2 es **imperativo** o **declarativo**? Justifica tu respuesta en 3 líneas como máximo.

PARADIGMA ORIENTADO A OBJETOS

Definición: es un estilo que trata los datos como objetos. En lugar de diseñar una lista gigante de instrucciones, diseña “objetos” con sus propias características y acciones.

Ejemplo de lenguaje: JAVA, C++ y Python son ejemplos de este paradigma. Python también asume programación funcional, así como JAVA 8. En el caso de C++ usa también la programación estructurada y genérica.

Trozo de código: Ejemplo en Python

```
class Coche:
    def     _init_(self, marca, color):
            self.marca = marca
    self.color = color
    self.encendido = False
def arrancar(self):
    self.encendido = True
    print(f"El coche {self.marca} de color {self.color} ha arrancado." )
```

Direcciones web:

https://openaccess.uoc.edu/server/api/core/bitstreams/1c3b6167-e1d9-4953-b1c5-ca85bb663c74/content

https://blog.educacionit.com/programacion-orientada-a-objetos-vs-programacion-estructurada/

https://www.prometeo-fp.com/blog/programacion-orientada-a-objetos-poo-la-base-de-la-programacion-moderna

PARADIGMA ESTRUCTURADO

Definición: se basa en seguir un enfoque descendente para resolver un problema complejo, convirtiendo problemas en subproblemas (o módulos) más pequeños y sencillos con diferente funcionalidad y que pueden interactúar entre sí. Estos programas son secuenciales y requieren estructuras de control (ciclos, condiciones,...).

Ejemplo de lenguaje: BASIC, C y JAVA son lenguajes que usa este tipo de paradigmas. En el caso de JAVA es además un lenguaje orientado a objetos.

Trozo de código: Ejemplo de BASIC:

```
REM *** Programa de saludo ***
CLS
PRINT “¡Hola! ¿Cómo te llamas?”
INPUT NOMBRE$
PRINT “Mucho gusto, “; NOMBRE$
PRINT “¡Adiós!”
END
```

Direcciones web:

https://openaccess.uoc.edu/server/api/core/bitstreams/1c3b6167-e1d9-4953-b1c5-ca85bb663c74/content

https://www.youtube.com/watch?v=8A95ajXTRa4

https://es.scribd.com/document/606916350/3-1-lenguajes-de-programacion-estructurada

¿el pseudocódigo de las actividades 1 y 2 es imperativo o declarativo?

No, ya que en ellos se han usados algoritmos mediante instrucciones para decirle a la máquina **cómo** ha de realizar el trabajo, paso a paso. Al contrario del declarativo que busca el resultado sin definir lo pasos que ha de seguir la máquina.

3. Entrega

La práctica se debe elaborar **de forma individual**. Deben realizarse **todas** las actividades planteadas en este documento. Cada actividad debe comenzar en una página nueva.

Se entregará a través del aula virtual en **formato PDF**, utilizando **este mismo documento** como plantilla. No se aceptarán entregas fuera de plazo ni por email.

En la **portada** debe figurar **tu nombre**. Recuerda citar las fuentes web en la actividad 3.

Debes escribir las respuestas a las actividades en **color azul**.